Calepin: Computational notebooks in Typst

This notebook demonstrates the *Calepin* executable Typst workflow: code chunks and inline snippets are collected during preprocessing, run with the requested engine, and rendered back into the document alongside the source that produced them.

Language-specific settings

`#calepin.setup()` sets document-wide defaults for all chunks unless overridden per chunk.

```
print(40 + 2)
```

```
42
```

Define a short alias for inline Python expressions:

Python

Python is a general-purpose programming language; learn more at python.org.

Start with a visible chunk when readers should see both the source and the captured output. This is the default notebook style and is useful while building up an example step by step.

```
print("#strong[42]")
```

```
#strong[42]
```

When the generated result should become part of the Typst document, set `results` to `"typst"`. Hiding the source lets the rendered document read like authored Typst while still keeping the value produced by the chunk.

42 in Typst

Inline snippets are for values that belong inside a sentence. They keep prose and executable code close together without introducing a full block.

The inline Python result is 42.

For graphical output, put the label and caption on the chunk itself. This keeps the figure metadata beside the code that creates the figure.

```
from plotnine import ggplot, aes, geom_point, labs
from plotnine.data import mtcars

(
    ggplot(mtcars, aes("mpg", "hp"))
    + geom_point(color = "orange")
    + labs(x="Miles per gallon", y="Horsepower")
).show()
```

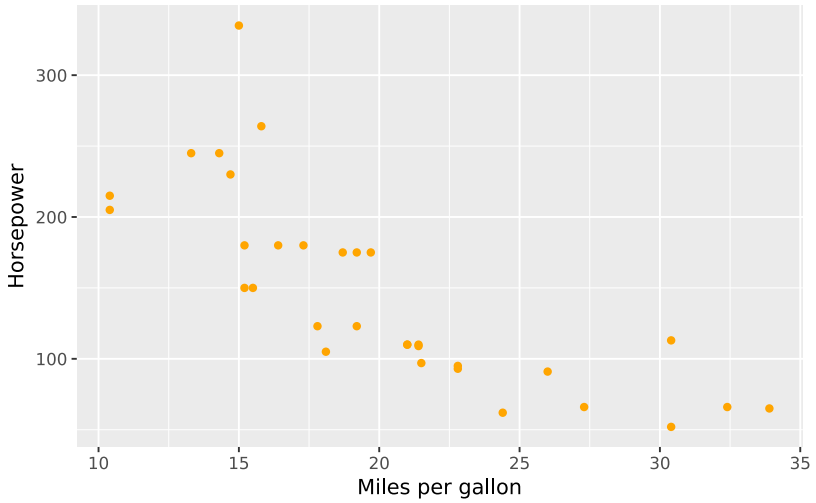

Figure 1: Plotnine scatterplot

R

R is a language and environment for statistical computing; learn more at r-project.org.

The R interface follows the same pattern as the Python interface. Use inline snippets for compact values that should appear directly in the surrounding paragraph.

The inline R result is 42.

Use a regular chunk when the source and captured console output should be shown together. The document structure stays the same even though the execution engine changes.

```r
mod <- lm(hp ~ mpg, data = mtcars)
summary(mod)
```

```
Call:
lm(formula = hp ~ mpg, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-59.26 -28.93 -13.45  25.65 143.36

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   324.08      27.43  11.813 8.25e-13 ***
mpg            -8.83       1.31  -6.742 1.79e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 43.95 on 30 degrees of freedom
Multiple R-squared:  0.6024,  Adjusted R-squared:  0.5892
F-statistic: 45.46 on 1 and 30 DF,  p-value: 1.788e-07
```

Chunks that produce graphics can carry the same label and caption fields. This makes figures portable across engines and keeps the rendered output easy to reference later.

```r
plot(hp ~ mpg, data = mtcars)
```

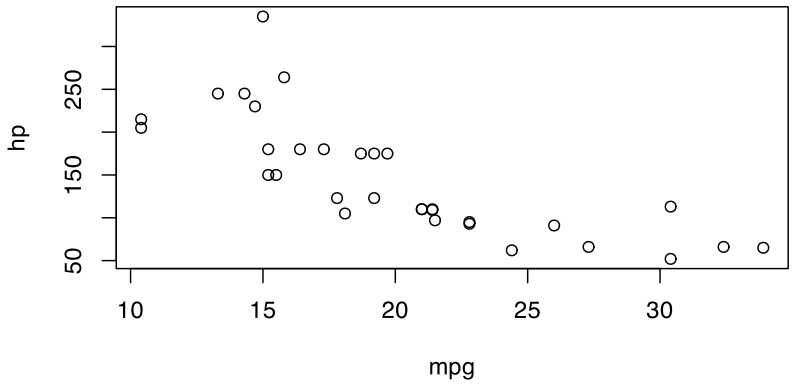

Figure 2: Scatterplot

Julia

Julia is a language for technical and scientific computing; learn more at julialang.org.

Julia runs through a Jupyter kernel. Use the kernel name reported by `jupyter kernelspec list`; this example uses `julia-1.12`.

The inline Julia result is 42.

For block output, pass the Jupyter kernel name as the chunk engine.

```
x = 40
println(x + 2)
```

```
42
```

Shell

Shell chunks run through the Bash Jupyter kernel. They behave like the other engines: the command is collected, executed, and its captured output is placed back into the document.

```
printf "hello from bash\n"
```

```
hello from bash
```

Mermaid

Mermaid is a text-based diagramming tool; learn more at mermaid.js.org.

Diagram engines use chunk bodies too, but the body is diagram source instead of program output. Give the chunk a label and caption when the rendered diagram should appear as a figure in the document.

```
%%{init: {"htmlLabels": false}}%%
flowchart LR
  Source["Typst document"] --> Query["Calepin query"]
  Query --> Execute["Run chunks"]
  Execute --> Render["Typst render"]
```

Figure 3: Mermaid flowchart

Graphviz DOT

Graphviz DOT is a graph description language used by Graphviz; learn more at graphviz.org.

The DOT engine follows the same figure pattern as Mermaid. Keeping the graph source in the Typst file makes the diagram editable alongside the prose that introduces it.

```
digraph {
  rankdir=LR
  Draft -> Review -> Publish
  Review -> Draft [label="revise"]
}
```

Figure 4: Graphviz state graph

TikZ

TikZ is a LaTeX package for creating vector graphics; learn more at tikz.dev.

TikZ chunks are another way to keep graphics source-controlled with the document. Use this pattern when a diagram is best authored in LaTeX-style drawing commands but should still be rendered through Calepin.

```
\begin{tikzpicture}
  \draw[thick, blue] (0,0) -- (2,1) -- (4,0);
  \fill[red] (2,1) circle (2pt);
\end{tikzpicture}
```

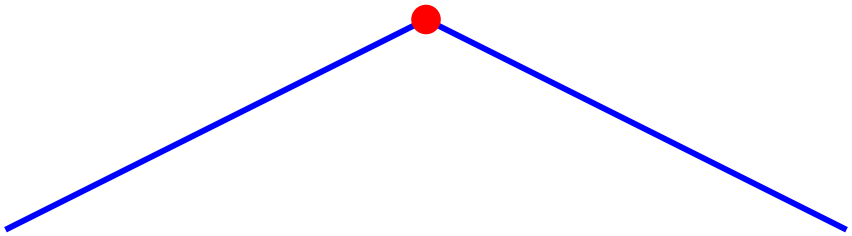

Figure 5: TikZ path

D2

D2 is a text-to-diagram language; learn more at d2lang.com.

D2 chunks demonstrate the same external-renderer workflow with a different diagram syntax. The important shape is unchanged: source in the chunk, rendered figure in the document.

```
direction: right

client -> api: request
```

```
api -> worker: job
worker -> database: write
```

Figure 6: D2 service sketch

Math

Currently, math is only supported by Typst in PDF output.

Use regular Typst math when the expression should be typeset by the document renderer instead of executed by a language engine. These examples show the math syntax living next to executable chunks in the same notebook.

$$A = \pi r^2$$

$$\text{area} = \pi \cdot \text{radius}^4$$

$$\mathcal{A} := \{x \in \mathbb{R} \mid x \text{ is natural}\}$$

$$5 < 17$$

Warning: On 2026-06-06, math export in HTML was only supported in the development version of Typst, available from Github.